MODULE

28

Web capstone: taskboard

Auth, SQL, permissions, live updates, tests and Docker, all in one API you can show in interviews.

YOU WILL LEARN

The project	`taskboard/`
Architecture	`taskboard/README.md`
Tests	`taskboard/tests/`
Challenges	`ex28_web_capstone_challenges.md`
Final exam	`QUIZ.md`

WHY CAPSTONE

A portfolio project beats a certificate

THE PROBLEM

Interviewers ask: **”Show me something you built.”** A to-do list from a tutorial doesn’t stand out. A secure, tested, deployed API with real-time updates does.

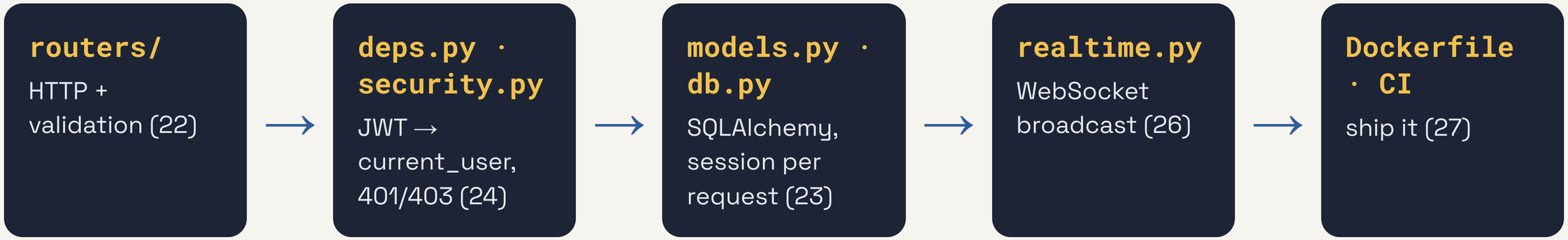
THE IDEA

taskboard uses every web module. Understand it, extend it, deploy it, and talk about its design.

PICTURE IT ARCHITECTURE

How taskboard fits together

📐 A request travels through the same layers as in the core capstone, plus the web pieces: auth, permissions, a database session per request, and a WebSocket hub that pushes changes to every open browser.



STEP BY STEP ARCHITECTURE

Files and the module they come from

FILE	JOB	MODULE
`config.py`	TASKBOARD_* env vars via BaseSettings	22, 27
`db.py` · `models.py`	engine, session per request, User 1---< Task	23
`schemas.py`	Pydantic in/out models	22
`security.py` · `deps.py`	scrypt hashes, JWT, current_user	24
`routers/tasks.py`	CRUD, owner-or-admin checks, pagination	22-24
`realtime.py`	push every change over WebSockets	26
`tests/` · `Dockerfile`	pytest, mypy --strict, ruff; container	16, 27

TRY IT PERMISSIONS

Owner-or-admin, the heart of the API

```python
from fastapi import HTTPException

def can_edit(task, user):
    return task.owner_id == user.id or user.role == "admin"

def update_task(task, user, changes):
    if not can_edit(task, user):
        raise HTTPException(404, "task not found")   # don't leak existence
    for field, value in changes.items():
        setattr(task, field, value)
    return task

class Obj:
    def __init__(self, **kw): self.__dict__.update(kw)

task = Obj(id=1, owner_id=7, title="Write slides", done=False)
owner, other, admin = Obj(id=7, role="user"), Obj(id=8, role="user"), Obj(id=1, role="admin")
print(update_task(task, owner, {"done": True}).done)
try:
    update_task(task, other, {"title": "hacked"})
except HTTPException as e:
    print(e.status_code, e.detail)
print(update_task(task, admin, {"title": "Write ALL slides"}).title)
```

OUTPUT

```
True
404 task not found
Write ALL slides
```

1. Authorisation in **one** function, tested on its own.
2. **404** instead of 403 hides other users' task IDs.
3. The real code is in **routers/tasks.py**.

STEP BY STEP QUALITY GATE

What ”done” means in this project

CHECK	COMMAND	WHY
Tests	`pytest`	auth, permissions, CRUD, WebSocket messages
Types	`mypy --strict`	no None bugs, typed dependencies
Lint + format	`ruff check` · `ruff format`	consistent, no smells
Container	`docker build`	runs the same everywhere
Docs	`/docs` + README	anyone can try the API in 2 minutes

IN REAL JOBS CAPSTONE

Make it yours: the challenges

Level 1

```
due dates + an admin view
filter: ?done=false&sort=-due
```

Level 2

```
Alembic migrations · login rate limit
Redis counters for tasks per user
```

Level 3

```
projects with members (many-to-many)
without N+1 queries
```

Level 3

```
per-user WebSocket channels
deploy to Azure Container Apps
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Tell me about a backend project you built.”

- **What and why**: a task API with accounts, permissions and live updates.
- **Stack**: FastAPI, SQLAlchemy, JWT auth, WebSockets, Docker, GitHub Actions.
- **A design decision**: owner-or-admin checks centralised in one dependency; 404 instead of 403 to avoid leaking IDs.
- **A problem you solved**: e.g. N+1 queries when listing tasks with owners, fixed with selectinload and a query-count test.
- **Quality**: tests, mypy --strict, ruff in CI. **Next**: Postgres, Alembic, rate limiting, deployment.

> **Follow-up they ask next:** ”How would you scale it to 1 million users?” (stateless replicas behind a load balancer, Postgres with indexes and read replicas, Redis cache and pub/sub for WebSockets, background workers, CDN).

RECAP REMEMBER THESE

Web track done: what you can do now

1. Explain HTTP and build pages with Flask/Jinja2.
2. Design typed, validated APIs with FastAPI and dependency injection.
3. Use SQLAlchemy and Django ORM without N+1; migrate safely.
4. Secure apps: hashing, JWT, permissions, OWASP defences.
5. Add real-time, background jobs and caching.
6. Containerise, test in CI, deploy to Azure.

PRACTICE NOW

Pick 2 challenges from ex28_web_capstone_challenges.md, deploy taskboard, and put the link on your CV and LinkedIn.